

Feature Selection and Comparison of different classifiers for the case of binary class response variables

Amartya Kumar Maulik Roll No.- 191010

June 21, 2026

Introduction

In general, there are two types of data mining – **Predictive** and **Descriptive**.

Predictive data mining will use variables or fields from a data-set to predict unknown or future values. The goal is to produce a model from a defined data-set that is used to perform classification, estimation, and other data mining tasks. On the other hand, descriptive data mining will focus on finding patterns that describe the data and can be interpreted by data analysts or business users. The goal is to gain understanding of the data by uncovering new relationships, patterns, and important information regarding the data-set.

Data mining can be valuable in virtually any industry and business scenario.

Objective

In this project , we will perform **Predictive Data Mining**. Our aim is to focus on the two of the most important aspects of data-mining viz. **Feature Selection** and **Model Selection**.

My objective is to determine the model providing the best results .We also have to keep in check on the fact that our selected model does not over-fit too much. Besides, this model selection , we shall also try to shed some light into processes that are used to select the features for our model based on the data-set of our choice.

Data Description

Title: Mushrooms

Source Information: UCI Machine Learning repository(Donated to UCI ML 27 April 1987)

Relevant Information: This dataset includes descriptions of hypothetical samples corresponding to 23 species of gilled mushrooms in the Agaricus and Lepiota Family Mushroom drawn from The Audubon Society Field Guide to North American Mushrooms (1981). Each species is identified as definitely edible, definitely poisonous, or of unknown edibility and not recommended. This latter class was combined with the poisonous one. The Guide clearly states that there is no simple rule for determining the edibility of a mushroom; no rule like "leaflets three, let it be" for Poisonous Oak and Ivy.

Number of Instances:8124

Number of Attributes:22 total (all of them are nominal)

Attribute Information:

There are two classes: **edible=e**, **poisonous=p**

Attribute	Attribute Range
cap-shape	bell=b,conical=c,convex=x,flat=f, knobbed=k,sunken=s
cap-surface	fibrous=f,grooves=g,scaly=y,smooth=s
cap-color	brown=n, buff=b, cinnamon=c, gray=g, green=r, pink=p, purple=u, red=e, white=w, yellow=y
bruises	bruises=t, no bruises=f
odor	almond=a, anise=l, creosote=c, fishy=y, foul=f, musty=m, none=n, pungent=p, spicy=s
gill-attachment	attached=a, descending=d, free=f, notched=n
gill-spacing	close=c, crowded=w, distant=d
gill-size	broad=b, narrow=n
gill-color	black=k, brown=n, buff=b, chocolate=h, gray=g, green=r, orange=o, pink=p, purple=u, red=e, white=w, yellow=y
stalk-shape	enlarging=e, tapering=t
stalk-root	bulbous=b, club=c, cup=u, equal=e, rhizomorphs=z, rooted=r, missing=?
stalk-surface-above-ring	fibrous=f, scaly=y, silky=k, smooth=s
stalk-surface-below-ring	fibrous=f, scaly=y, silky=k, smooth=s
stalk-color-above-ring	brown=n, buff=b, cinnamon=c, gray=g, orange=o, pink=p, red=e, white=w, yellow=y
stalk-color-below-ring	brown=n, buff=b, cinnamon=c, gray=g, orange=o, pink=p, red=e, white=w, yellow=y
veil-type	partial=p, universal=u
veil-color	brown=n, orange=o, white=w, yellow=y
ring-number	none=n, one=o, two=t
ring-type	cobwebby=c, evanescent=e, flaring=f, large=l, none=n, pendant=p, sheathing=s, zone=z
spore-print-color	black=k, brown=n, buff=b, chocolate=h, green=r, orange=o, purple=u, white=w, yellow=y
population	abundant=a, clustered=c, numerous=n, scattered=s, several=v, solitary=y
habitat	grasses=g, leaves=l, meadows=m, paths=p, urban=u, waste=w, woods=d

```
In [736]: df=pd.read_csv ('datasets_478_974_mushrooms.csv')
df.head()
```

```
Out[736]:
```

	class	cap-shape	cap-surface	cap-color	bruises	odor	gill-attachment	gill-spacing	gill-size	gill-color	...	stalk-surface-below-ring	stalk-color-above-ring	stalk-color-below-ring	veil-type	veil-color	ring-number	ring-type	spore-print-color	population
0	p	x	s	n	t	p	f	c	n	k	...	s	w	w	p	w	o	p	k	s
1	e	x	s	y	t	a	f	c	b	k	...	s	w	w	p	w	o	p	n	n
2	e	b	s	w	t	l	f	c	b	n	...	s	w	w	p	w	o	p	n	n
3	p	x	y	w	t	p	f	c	n	n	...	s	w	w	p	w	o	p	k	s
4	e	x	s	g	f	n	f	w	b	k	...	s	w	w	p	w	o	e	n	a

5 rows x 23 columns

```
In [737]: df.columns
```

```
Out[737]: Index(['class', 'cap-shape', 'cap-surface', 'cap-color', 'bruises', 'odor',
      'gill-attachment', 'gill-spacing', 'gill-size', 'gill-color',
      'stalk-shape', 'stalk-root', 'stalk-surface-above-ring',
      'stalk-surface-below-ring', 'stalk-color-above-ring',
      'stalk-color-below-ring', 'veil-type', 'veil-color', 'ring-number',
      'ring-type', 'spore-print-color', 'population', 'habitat'],
      dtype='object')
```

Methodology

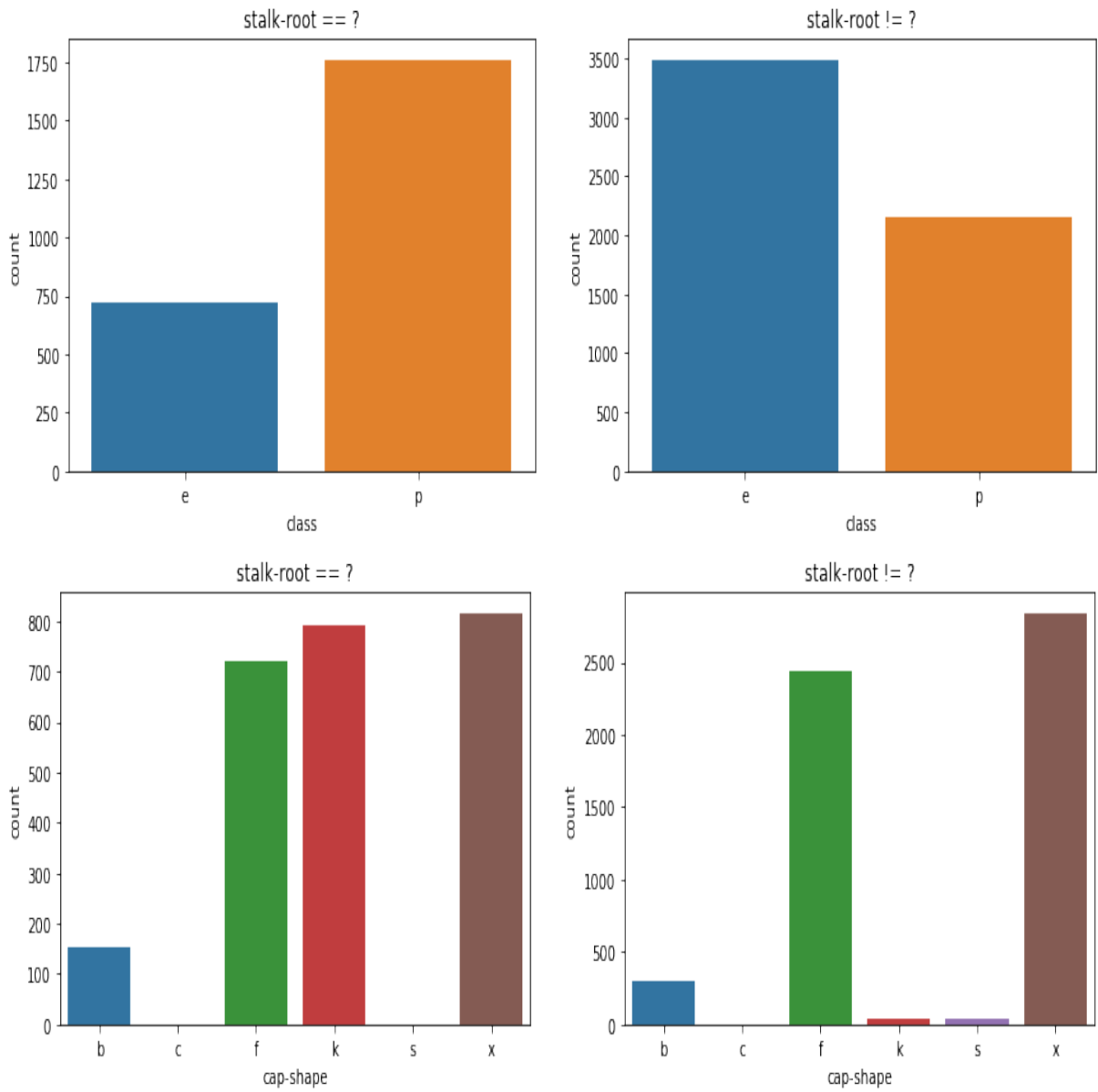
Before proceeding further , we need to have a clear road-map of how to approach the problems.This will help us in keeping us in the right track and reduce any human-induced errors which may led to other computational errors. We are going to perform the following steps in their order :

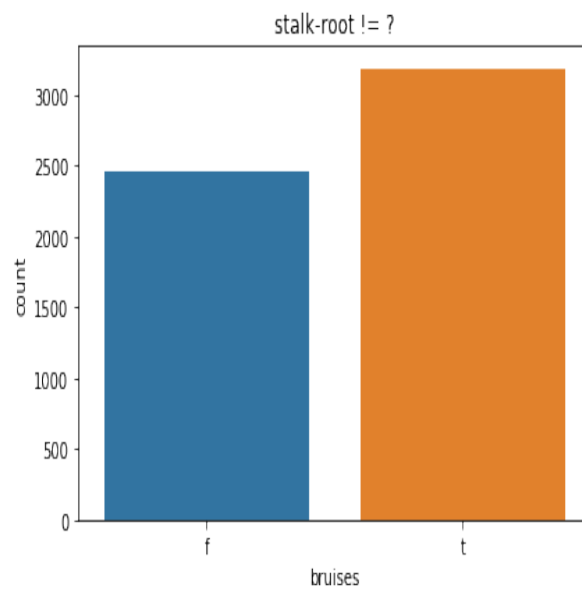
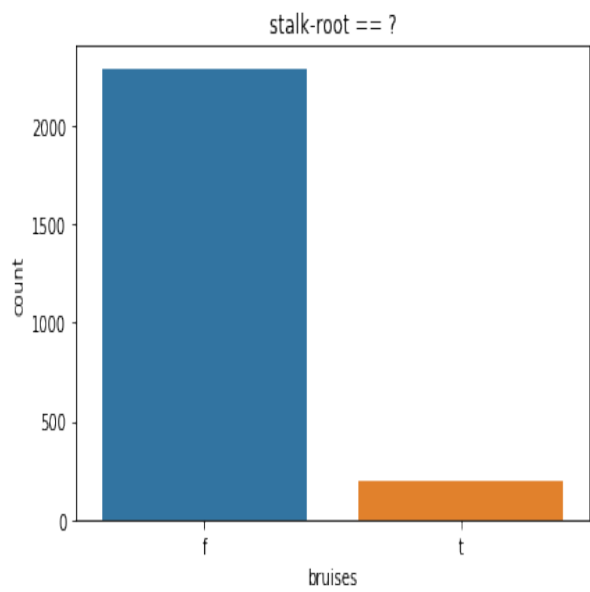
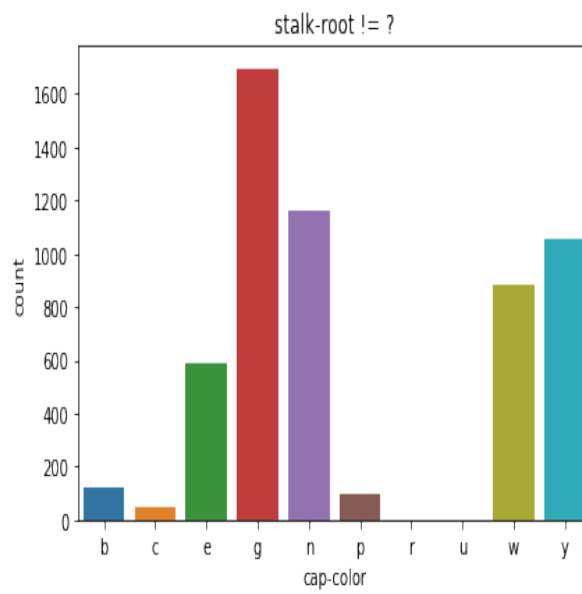
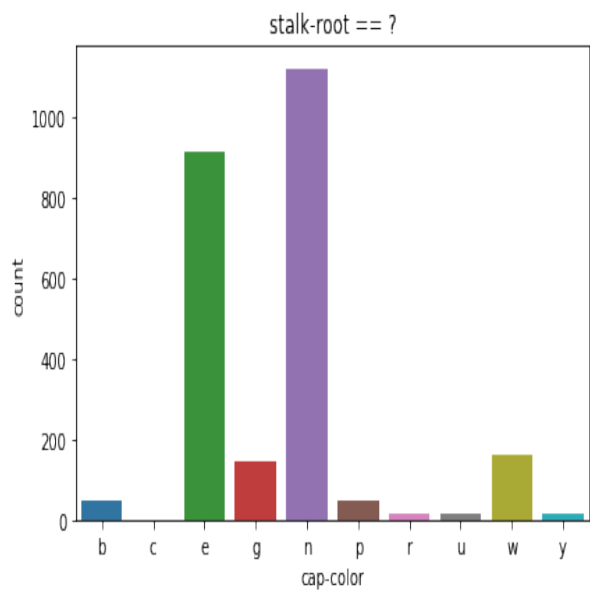
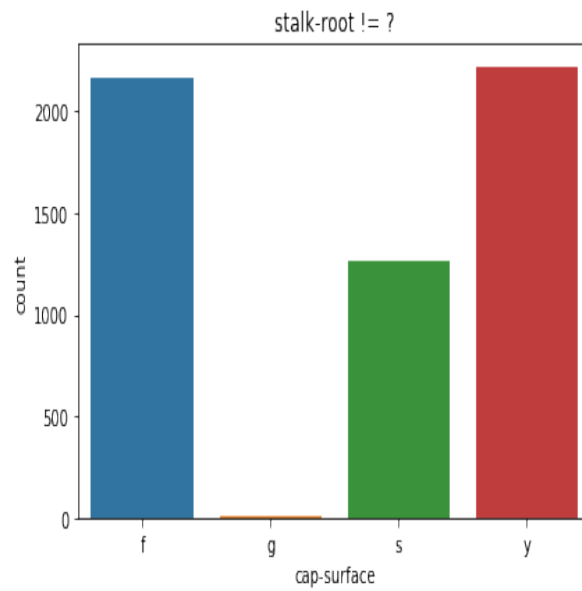
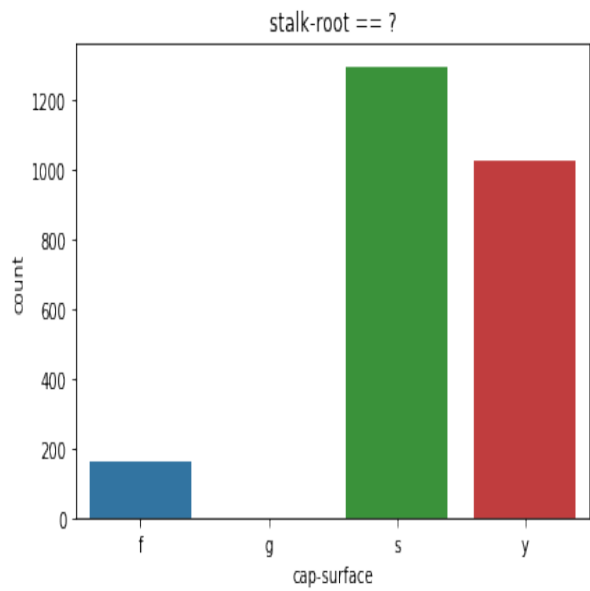
- Data Pre-Processing
 - Null or Missing Value Detection
 - Data Transformation
- Feature Selection
 - Removal of Constant and Duplicate Features
 - Selection of the Best Features
- Model Selection And Comparison

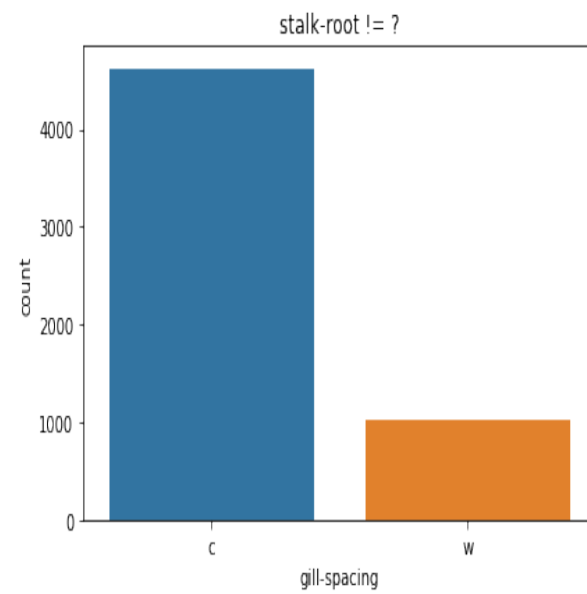
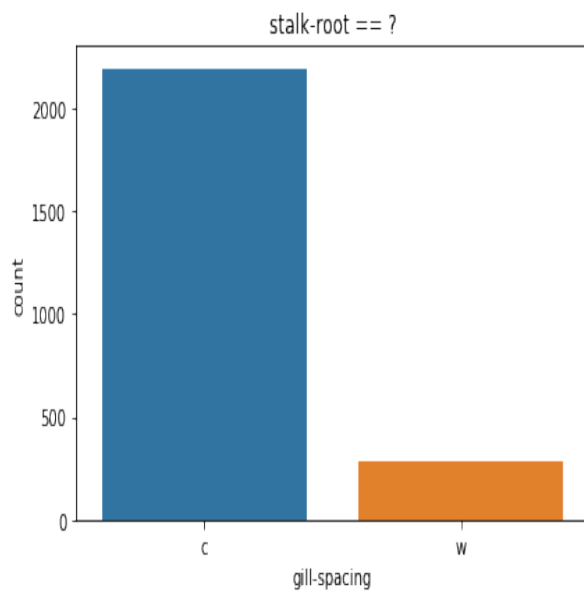
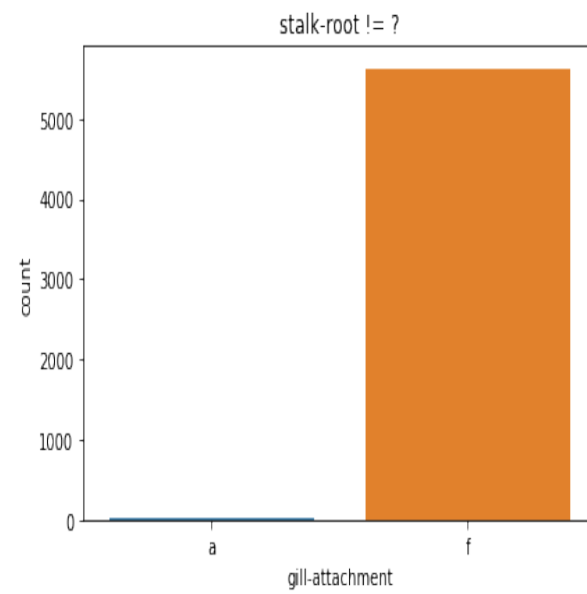
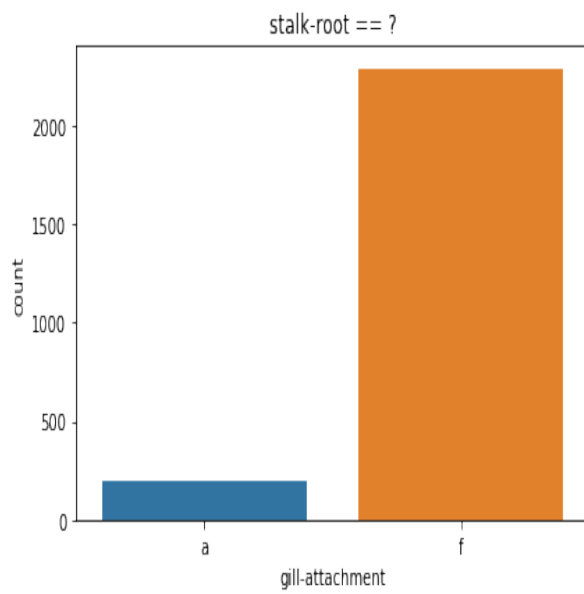
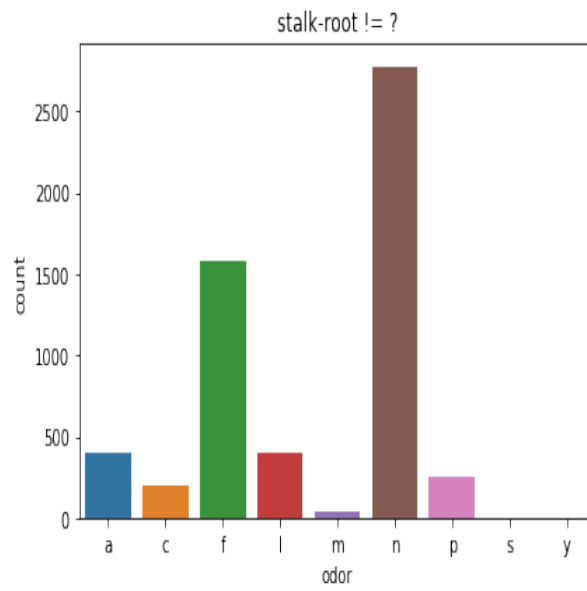
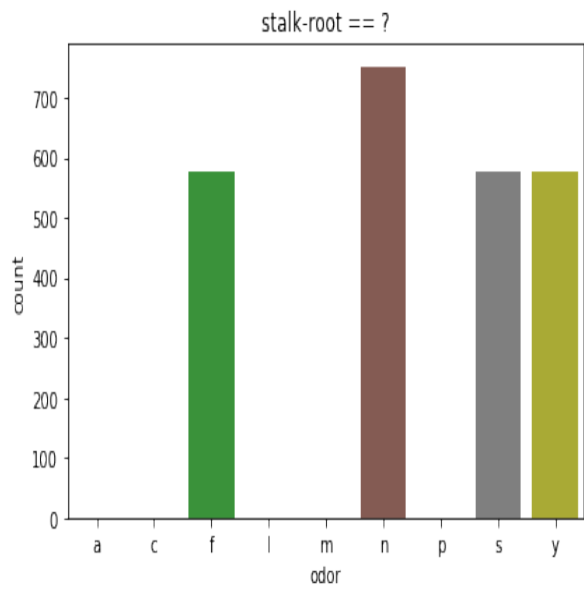
Data Pre-processing

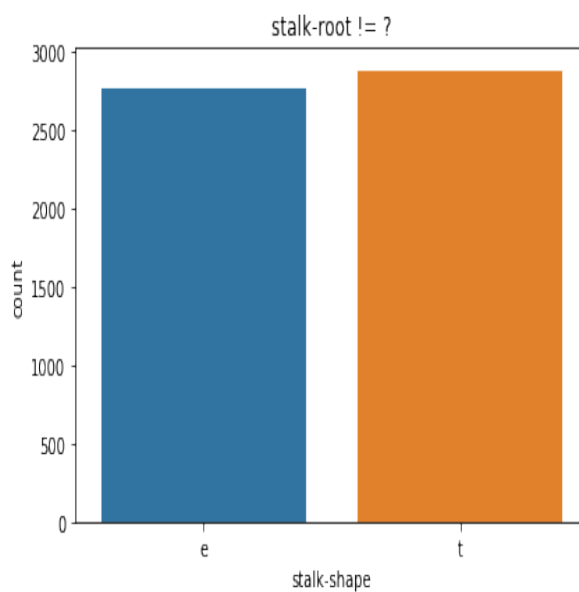
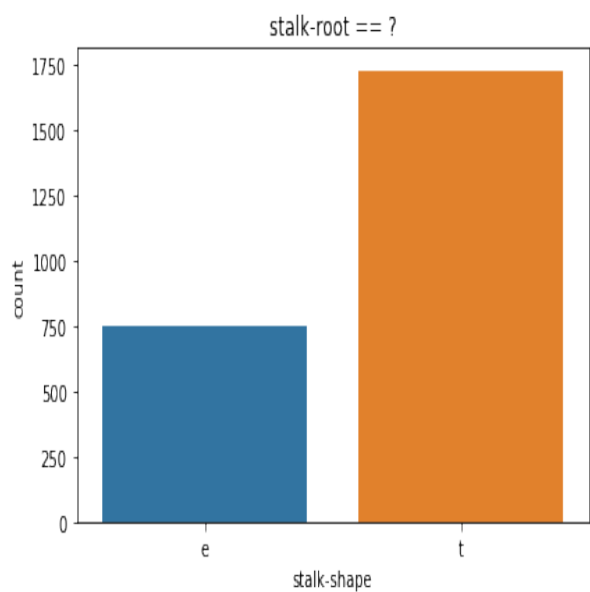
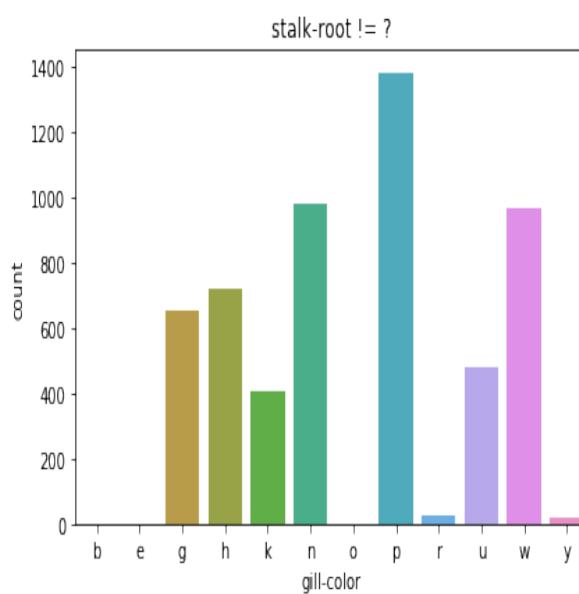
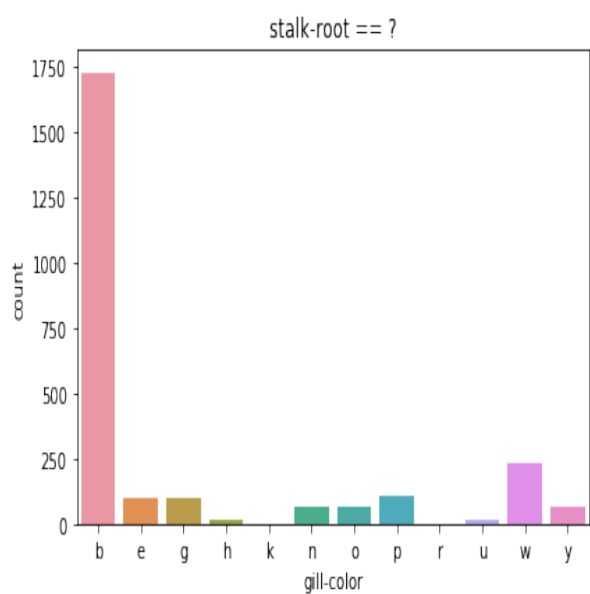
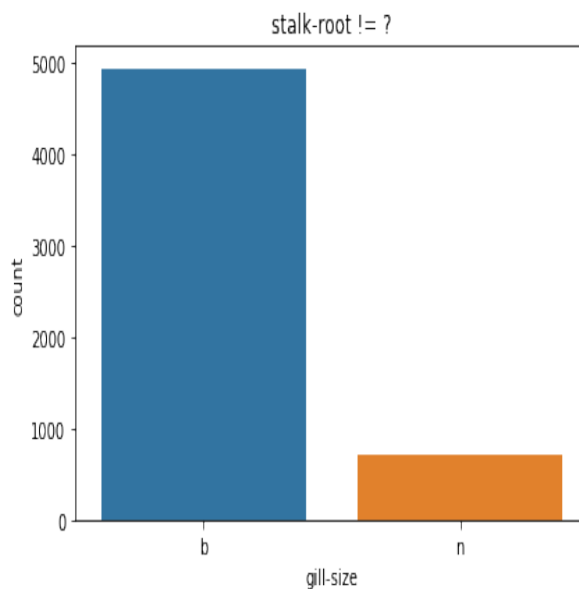
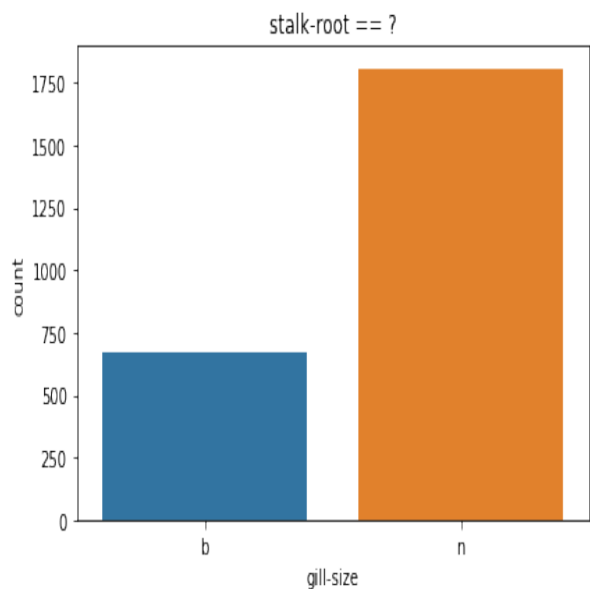
Null Detection

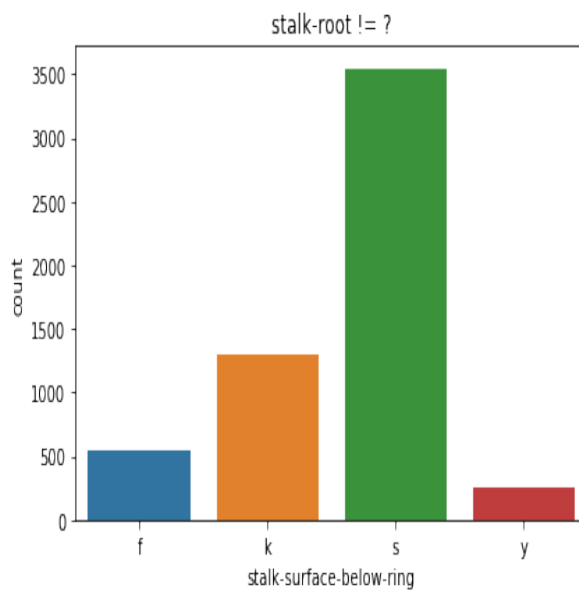
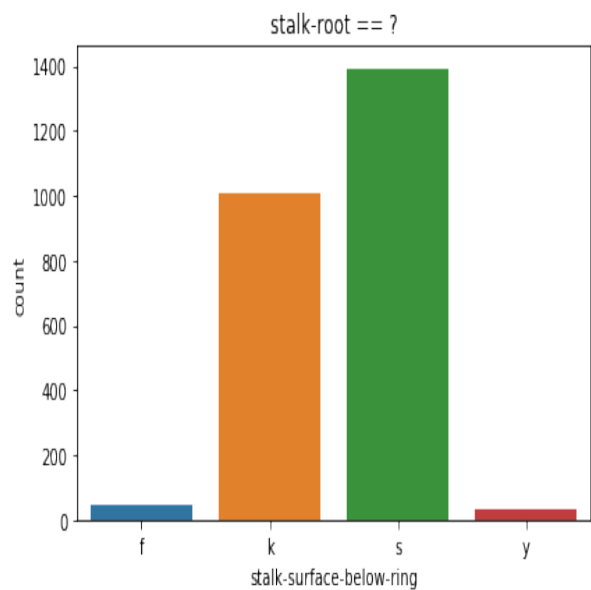
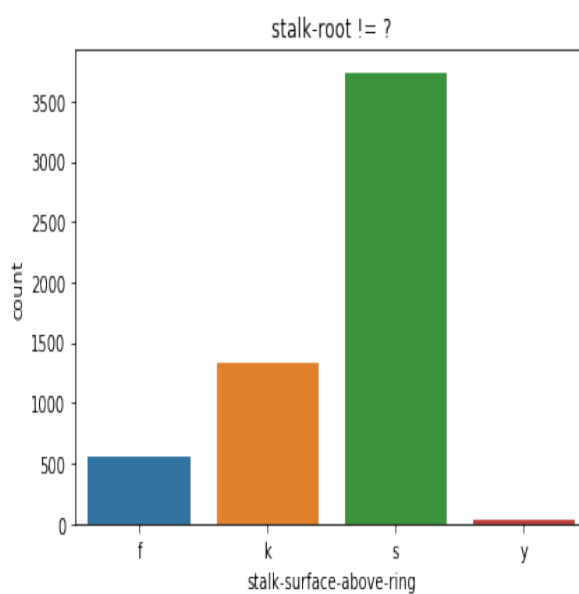
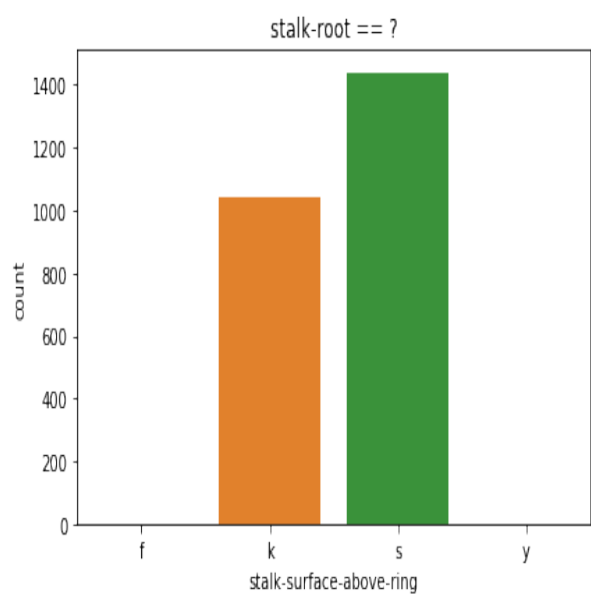
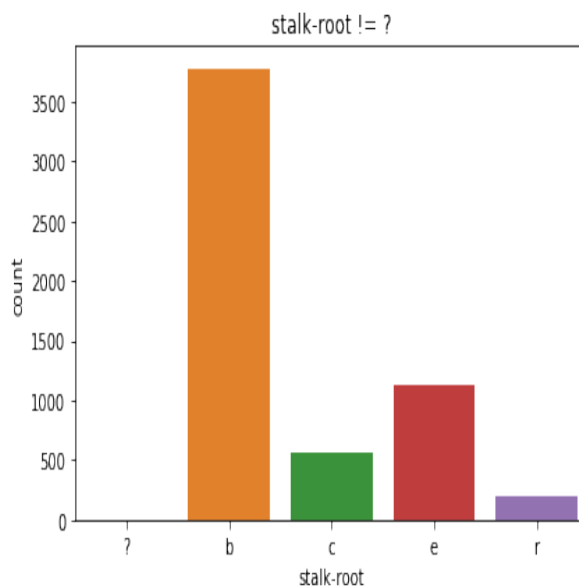
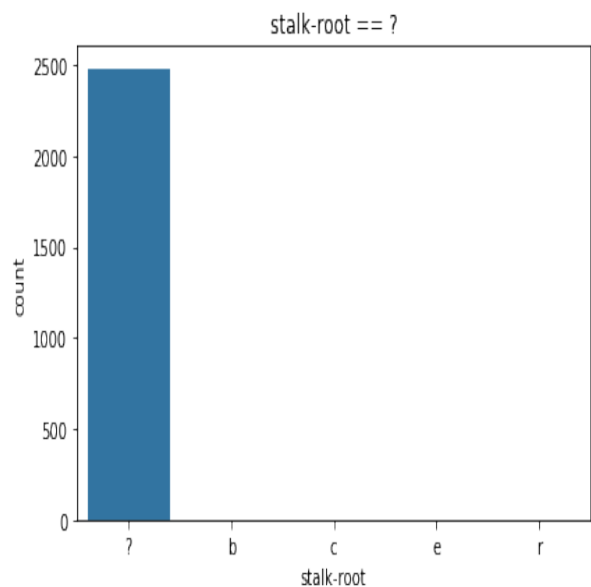
First of all , we have to check if there is any missing value present in the data. The stalk-root variable has a "?" value for it's missing values. Rather than impute this missing value, we will divide the dataset into two sections: (1) where $X[\text{'stalk-root'}] == ?$ and (2) where $X[\text{'stalk-root'}] != ?$. Then, we will analyze the distribution of each variable within those two data sets to determine if they are similar. we will expect that if the distributions vary greatly for each variable, then the fact that the stalk-root's are missing for some of the mushrooms and not missing for the others may turn out to be useful/relevant information.

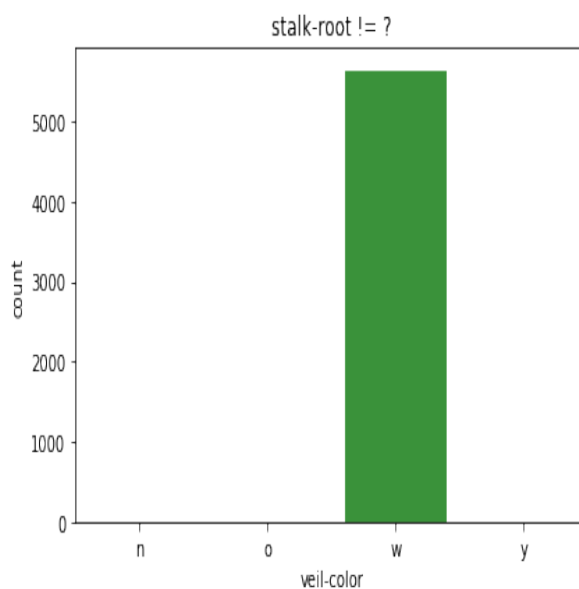
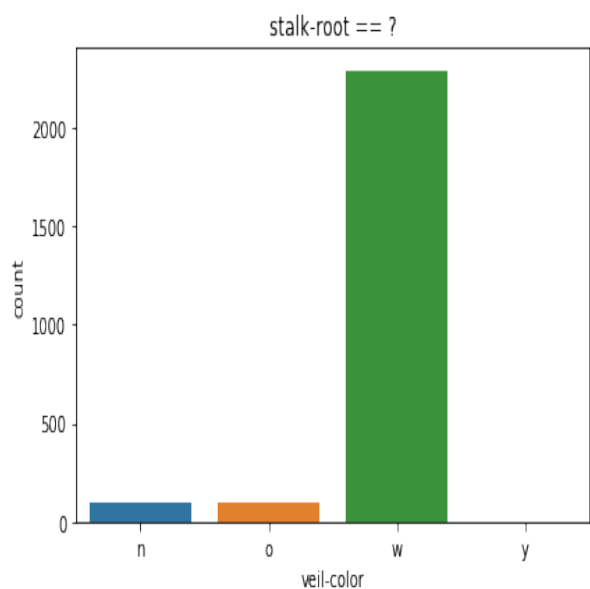
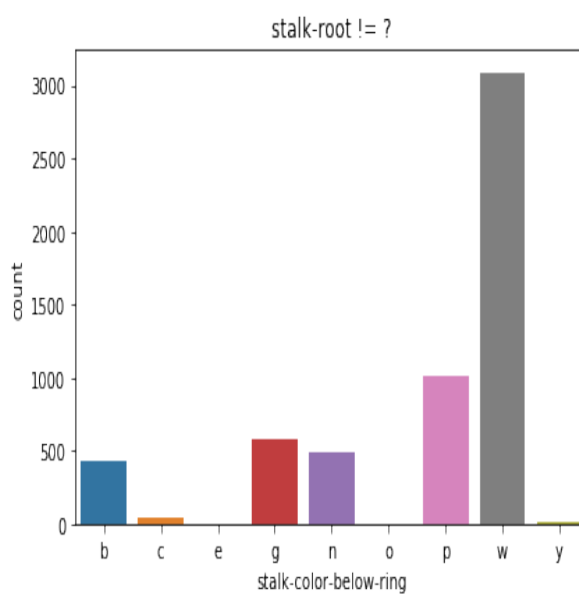
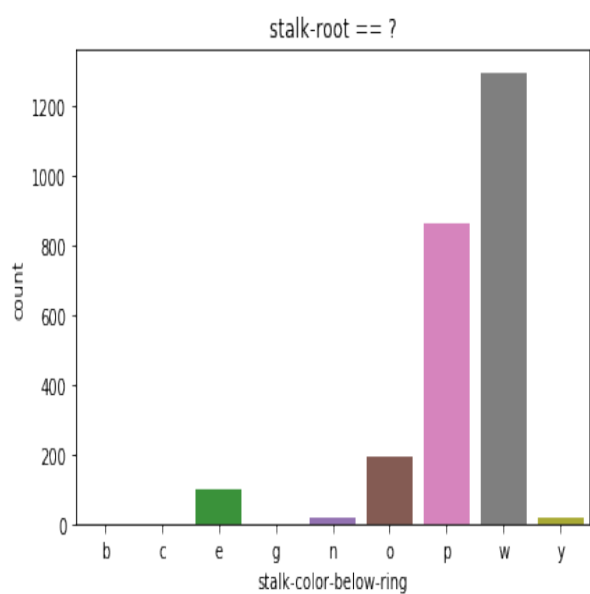
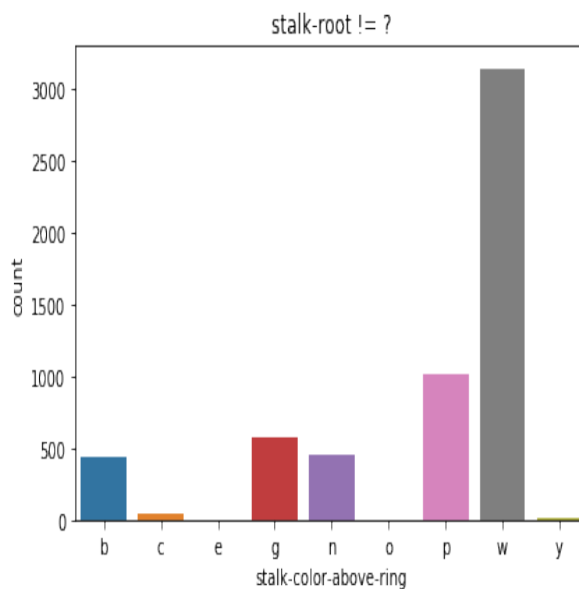
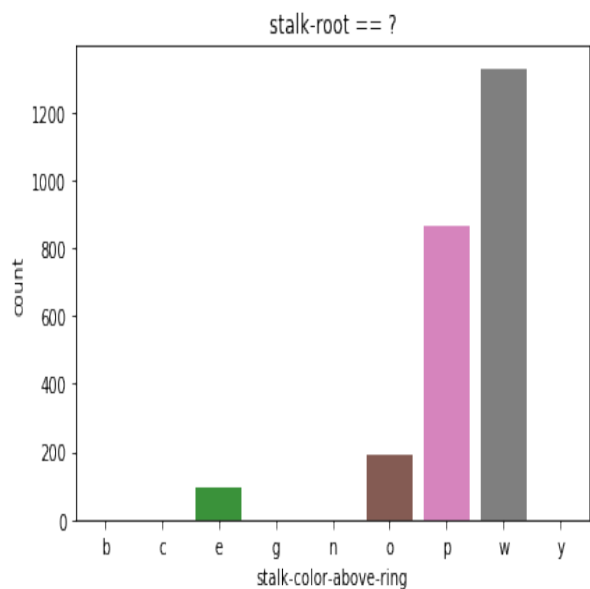


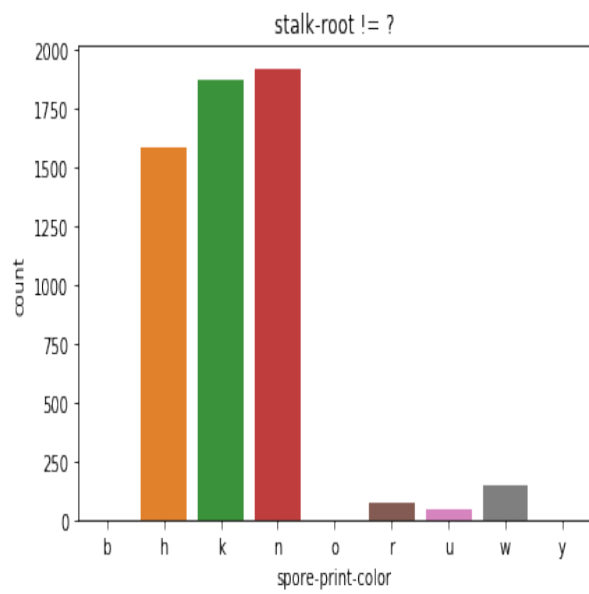
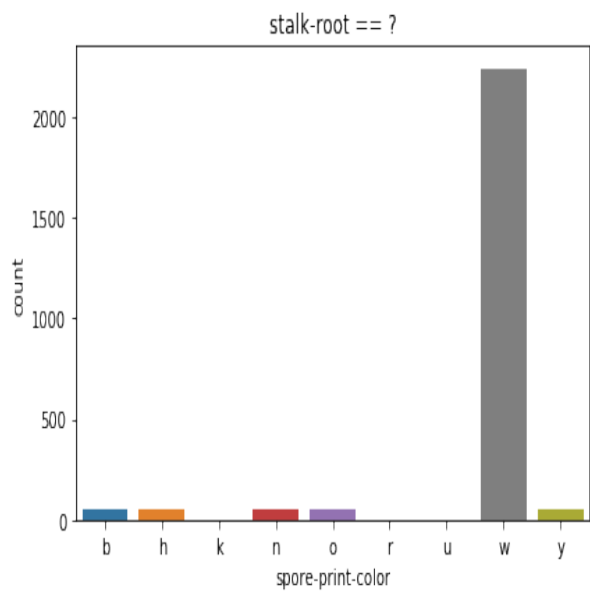
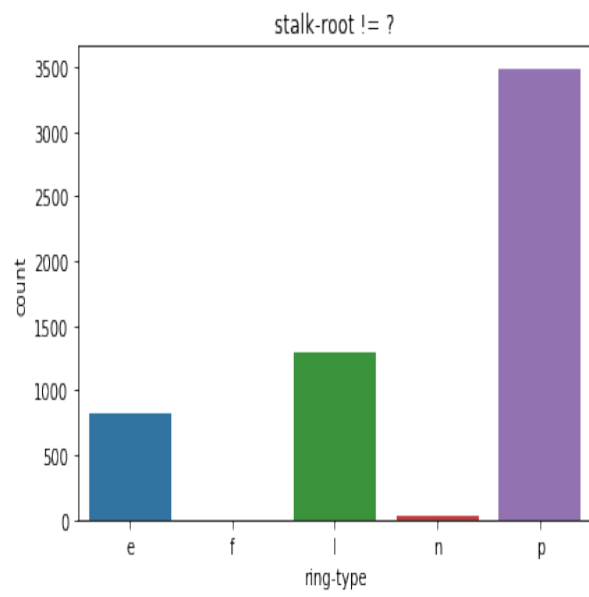
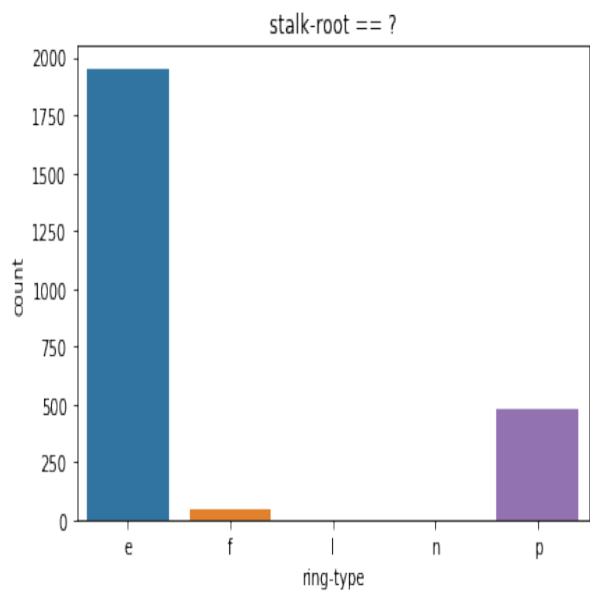
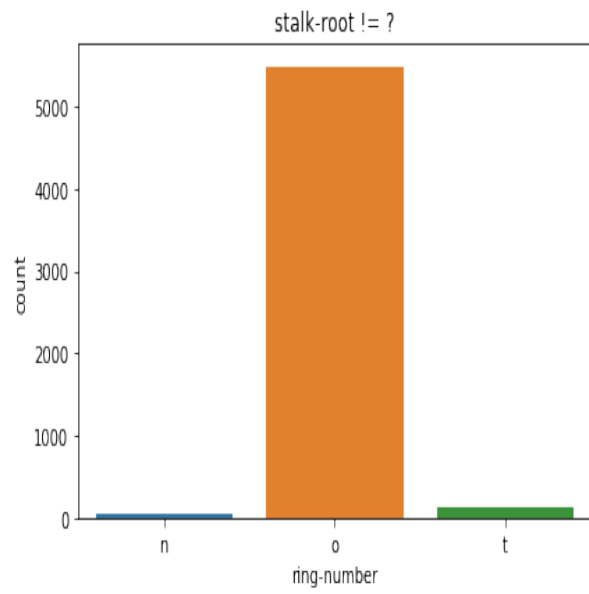
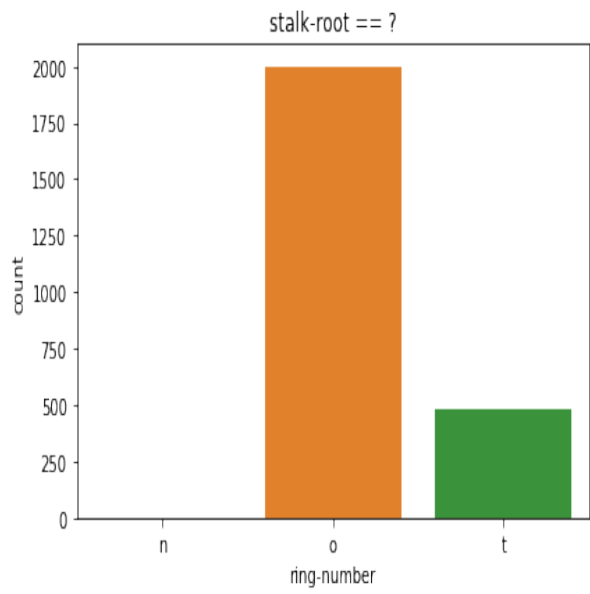


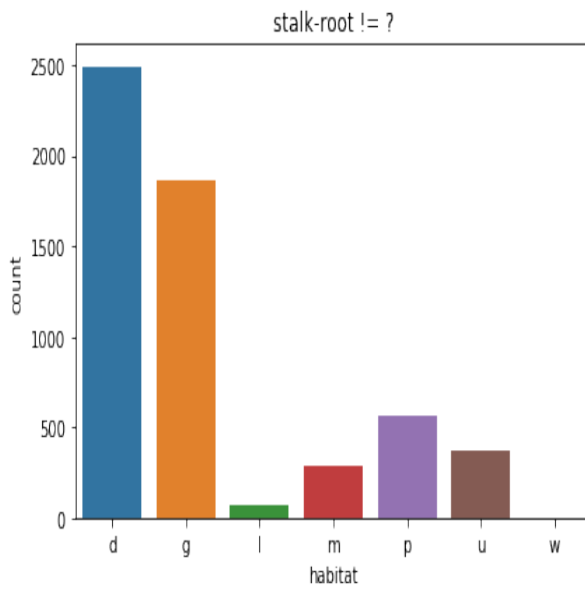
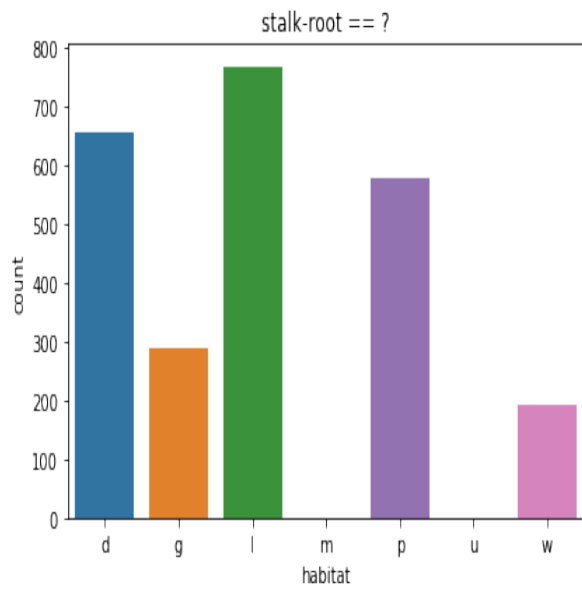
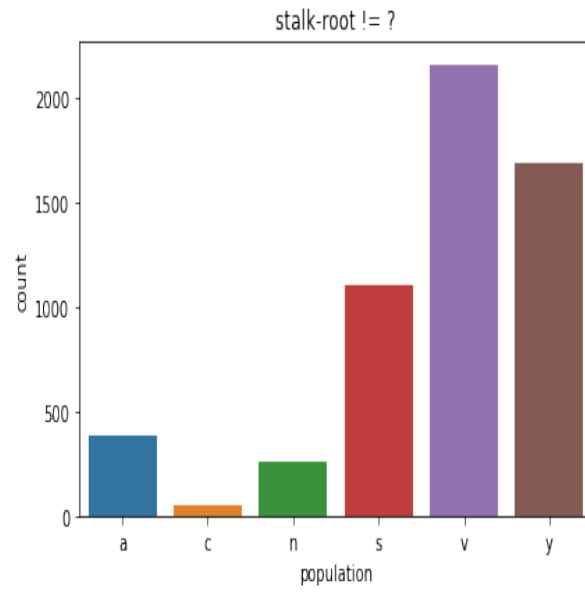
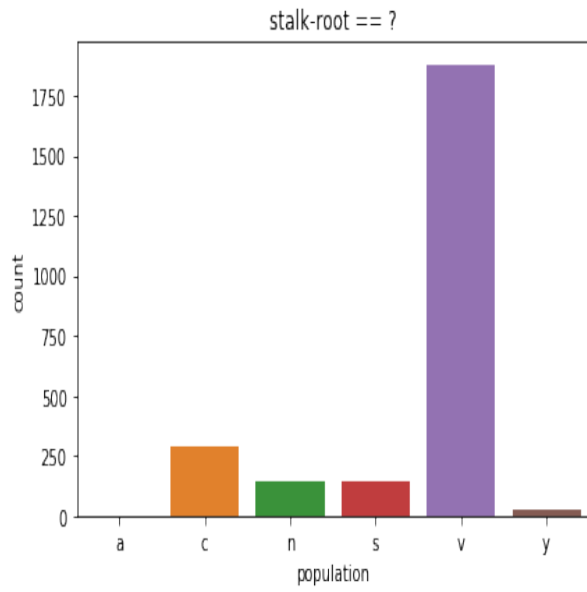












Since many of the distributions vary greatly, and because 30.526834071885773 % of the mushrooms have the value "?" for their stalk-root, we will not impute the "?" values, rather, we will encode them just as we would the rest of the values in that column.

```
In [739]: print( (len(df[df['stalk-root']=='?']) / len(X))*100, '%', sep='')
30.526834071885773%
```

```
In [740]: #This shows that there is no other null values
df.isnull().sum()
```

```
Out[740]: class                0
cap-shape                0
cap-surface              0
cap-color                0
bruises                  0
odor                     0
gill-attachment          0
gill-spacing             0
gill-size                0
gill-color               0
stalk-shape              0
stalk-root               0
stalk-surface-above-ring 0
stalk-surface-below-ring 0
stalk-color-above-ring   0
stalk-color-below-ring   0
veil-type                0
veil-color               0
ring-number              0
ring-type                0
spore-print-color        0
population               0
habitat                  0
dtype: int64
```

Thus , there are no null or missing value present in the data.

Imbalance Check

The output of the data set is of two categories: **edible=e** and **poisonous=p**. Thus we have to check if our data set is imbalanced or not.

Checking of imbalance in dataset

```
In [741]: # As the y takes both p(poisonous) and e(edible) (the two classes of the y present in the dataset) almost equally , we can
# conclude that there is not an imbalanced dataset.
df.groupby('class').size()
```

```
Out[741]: class
e      4208
p      3916
dtype: int64
```

So, the data-set shows no sign of imbalance.

Note: Imbalance in a data-set occurs when there is a exceptionally high presence of a particular output in the data-set.

Data Transformation

As all of the features are categorical, non-ordinal variable, we have to label them. But we can't just label them on the basis of uniqueness , i.e. suppose a feature has 4 categories , we can't just label them as 0,1,2 and 3 because the machine will think that the category that is labelled as 2 must have closer relationship with the category labelled as 3 instead of the category labelled as 0. This interpretation done by the machine is totally baseless as in reality that may not even the case. This means that it doesn't make sense to order them from largest to smallest using larger or small numbers. Data like this is generally better modelled using a one-hot encoding. So we create dummy variables for each of the feature to avoid this discrepancy. *Thus from 22 features we get a total of 95 dummy variables.*

We then put the features into a feature matrix X and the target or output into y .

We will split the data randomly into training set and test set. We are going keep aside 30 percent of the data to test our model. Thus our training data has 5686 instances and test data has 2438 instances.

```
In [733]: for col in df.columns:
          df = pd.get_dummies(df,prefix=col, columns = [col], drop_first=True)

          df.head()
```

Out[733]:

	class_p	cap- shape_c	cap- shape_f	cap- shape_k	cap- shape_s	cap- shape_x	cap- surface_g	cap- surface_s	cap- surface_y	cap- color_c	...	population_n	population_s	population_v	population
0	1	0	0	0	0	1	0	1	0	0	...	0	1	0	
1	0	0	0	0	0	1	0	1	0	0	...	1	0	0	
2	0	0	0	0	0	0	0	1	0	0	...	1	0	0	
3	1	0	0	0	0	1	0	0	1	0	...	0	1	0	
4	0	0	0	0	0	1	0	1	0	0	...	0	0	0	

5 rows x 96 columns

Data Reduction

Removal of Constant and Duplicate Features

The first thing that is required to be done is the reduction of the feature space to bring down our computational expense. Thus , we will remove the constant and duplicate features as they do not provide any new information. As we can see that the number of features has decreased by 5. The new train and test feature matrices are $(X_{train_1}, X_{test_1})$

Feature Selection

Feature selection helps to zone in on the relevant variables in a data set, and can also help to eliminate collinear variables. It helps reduce the noise in the data set, and it helps the model pick up the relevant signals.

Advantage:

- Reduces Over-fitting: Less redundant data means less possibility of making decisions based on redundant data/noise.
- Improves Accuracy: Less misleading data means modeling accuracy improves.
- Reduces Training Time: Less data means that algorithms train faster.

As all the features are categorical , a correlation matrix does not make any sense. Thus we cannot perform the usual principal component analysis (PCA).

Now we will consider feature selection based on two metrics and a model-fitting technique.

1.We will use the **chi-square test** to measure the importance of the features.A chi-square test is used in statistics to test the independence of two events.In feature selection, we aim to select the features which are highly dependent on the response.



The figure depicts that those features with high p-value is independent of the response and can not be considered for model training. There seems to be at least 8 features that may not contribute into model training if we consider the level of significance to be 0.05

Here, we consider the just the top 20 percentile of the features according to their importance to train our model.

2.We will use the **mutual information** to measure the importance of the features.Mutual information is a measure between two (possibly multi-dimensional) random variables X and y , that quantifies the amount of information obtained about one random variable, through the other random variable.When it comes to feature selection, we would like to maximise the mutual information between the subset of selected features $X_S(suppose)$ and the target variable y .

Here, we consider the just the top 20 percentile of the features according to their importance to train our model.

3.Now this is slightly different technique. We will select **the features based on logistic regression**.This algorithm will train the logistic regression model on various subsets of features and give the subset of features as output which results

in the best fit of the model.

Here , we consider a maximum of 20 features to train our model.

Using the above-mentioned methods of feature selection , we reduce our feature space. The new reduced feature train and test matrices are $(X_{train_2}, X_{test_2})$, $(X_{train_3}, X_{test_3})$, $(X_{train_4}, X_{test_4})$ using the first , second and third method respectively.

```
In [742]: #Feature selection based on three differnt metrics
def select_feat1(X_train , y_train ,X_test):
    fs=SelectPercentile(score_func=chi2,percentile=20)
    fs.fit(X_train ,y_train)
    X_train_fs=fs.transform(X_train)
    X_test_fs=fs.transform(X_test)
    return X_train_fs , X_test_fs

def select_feat2(X_train , y_train ,X_test):
    fs=SelectPercentile(score_func=mutual_info_classif ,percentile=20)
    fs.fit(X_train ,y_train)
    X_train_fs=fs.transform(X_train)
    X_test_fs=fs.transform(X_test)
    return X_train_fs , X_test_fs

def select_feat3(X_train , y_train , X_test):
    sel=SelectFromModel(LogisticRegression(penalty='l1', C=0.5 , random_state = 42,solver='liblinear' , n_jobs=-1 )
                        , max_features = 20 )
    sel.fit(X_train_1,y_train)
    sel.get_support()
    sel.estimator_.coef_
    X_train_l1=sel.transform(X_train_1)
    X_test_l1=sel.transform(X_test_1)
    return X_train_l1,X_test_l1

In [743]: X_train_2 , X_test_2 =select_feat1(X_train_1,y_train , X_test_1)
X_train_3 , X_test_3 =select_feat2(X_train_1,y_train , X_test_1)
X_train_4 , X_test_4 =select_feat3(X_train_1,y_train , X_test_1)
X_train_2.shape , X_train_3.shape ,X_train_4.shape
```

```
Out[743]: ((5686, 18), (5686, 18), (5686, 20))
```

Model Fitting and Comparison

Model Description

Here, We are going to use the following models:

1.Random Forest Classifier :It is an ensemble tree-based learning algorithm. The Random Forest Classifier is a set of decision trees from randomly selected subset of training set. It aggregates the votes from different decision trees to decide the final class of the test object.

Features and Advantages:

- It is one of the most accurate learning algorithms available. For many data sets, it produces a highly accurate classifier.
- It runs efficiently on large databases.
- It can handle thousands of input variables without variable deletion.
- It gives estimates of what variables that are important in the classification.
- It generates an internal unbiased estimate of the generalization error as the forest building progresses.
- It has an effective method for estimating missing data and maintains accuracy when a large proportion of the data are missing.

Disadvantages:

- Random forests have been observed to overfit for some datasets with noisy classification/regression tasks.
- For data including categorical variables with different number of levels, random forests are biased in favor of those attributes with more levels. Therefore, the variable importance scores from random forest are not reliable for this type of data.

2.Decision Tree Classifier:A Decision Tree is a simple representation for classifying examples. It is a Supervised Machine Learning where the data is continuously split according to a certain parameter.

Advantages:

- Inexpensive to construct.
- Extremely fast at classifying unknown records.
- Easy to interpret for small-sized trees
- Accuracy comparable to other classification techniques for many simple data sets.
- Excludes unimportant features.

Disadvantages:

- Easy to over-fit.
- Decision Boundary restricted to being parallel to attribute axes.
- Decision tree models are often biased toward splits on features having a large number of levels.
- Small changes in the training data can result in large changes to decision logic.
- Large trees can be difficult to interpret and the decisions they make may seem counter intuitive.

3.Logistic Regression: Logistic Regression is a Machine Learning algorithm which is used for the classification problems.It is a predictive analysis algorithm .Logistic regression transforms its output using the logistic sigmoid function to return a probability value.

Limitations: Logistic regression is a simple and powerful linear classification algorithm. It also has limitations that suggest at the need for alternate linear classification algorithms.

- Two-Class Problems : Logistic regression is intended for two-class or binary classification problems. It can be extended for multi-class classification, but is rarely used for this purpose.
- Unstable With Well Separated Classes : Logistic regression can become unstable when the classes are well separated.
- Unstable With Few Examples : Logistic regression can become unstable when there are few examples from which to estimate the parameters.

4.Linear Discriminant Analysis:

Linear Discriminant Analysis does address each of limitations of the Logistic regression and is the go-to linear method for multi-class classification problems. Even with binary-classification problems, it is a good idea to try both logistic regression and linear discriminant analysis.If you have more than two classes then Linear Discriminant Analysis is the preferred linear classification technique.The goal of LDA is to project the features in higher dimensional space onto a lower-dimensional space in order to avoid the curse of dimensionality and also reduce resources and dimensional costs.

```
In [745]: clf1=RandomForestClassifier(n_estimators=100,max_depth = 3 ,criterion="entropy",random_state=42 , n_jobs=-1 )
clf2=DecisionTreeClassifier(criterion="entropy",max_depth = 3 ,random_state=42 )
clf3=LogisticRegression(penalty='elasticnet', C=0.5 , random_state = 42,solver='saga',l1_ratio=0.5, n_jobs=-1 )
clf4=LinearDiscriminantAnalysis(solver = 'eigen' , shrinkage = 'auto')
```

```
In [746]: def run_randomforest(X_train , X_test , y_train , y_test):
    clf = clf1
    clf.fit(X_train , y_train)
    y_pred = clf.predict(X_test)
    print('Accuracy on test set:', accuracy_score(y_test , y_pred))
    print(confusion_matrix(y_test, y_pred))

def run_decisiontree(X_train , X_test , y_train , y_test):
    clf = clf2
    clf.fit(X_train , y_train)
    y_pred = clf.predict(X_test)
    print('Accuracy on test set:', accuracy_score(y_test , y_pred))
    print(confusion_matrix(y_test, y_pred))

def run_logistic(X_train , X_test , y_train , y_test):
    clf = clf3
    clf.fit(X_train , y_train)
    y_pred = clf.predict(X_test)
    print('Accuracy on test set:', accuracy_score(y_test , y_pred))
    print(confusion_matrix(y_test, y_pred))

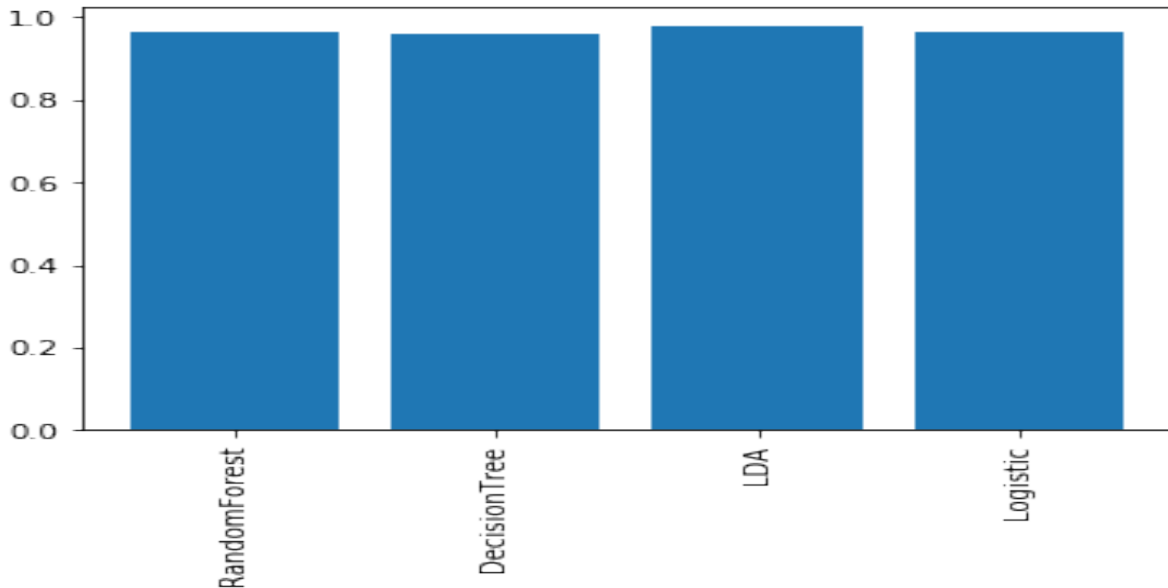
def run_LDA(X_train , X_test , y_train , y_test):
    clf = clf4
    clf.fit(X_train , y_train)
    y_pred = clf.predict(X_test)
    print('Accuracy on test set:', accuracy_score(y_test , y_pred))
    print(confusion_matrix(y_test, y_pred))
```

Selection of Feature-selection method

Firstly, we will cross-validate the models on the training set. since I am applying a classifier, **cross val score** splits the data into 5 almost equal sized pieces, trains on each combination of 4 and gives back the accuracy of the estimator on the 5th when **cv=5** is set .The **cross val score** estimates the expected accuracy of your model on out-of-training data (pulled from the same underlying process as the training data, of course). The benefit is that one need not set aside any data to obtain this metric, and you can still train your model on all of the available data.

There are 3 kinds of reduced feature matrices to train and cross-validate our model.

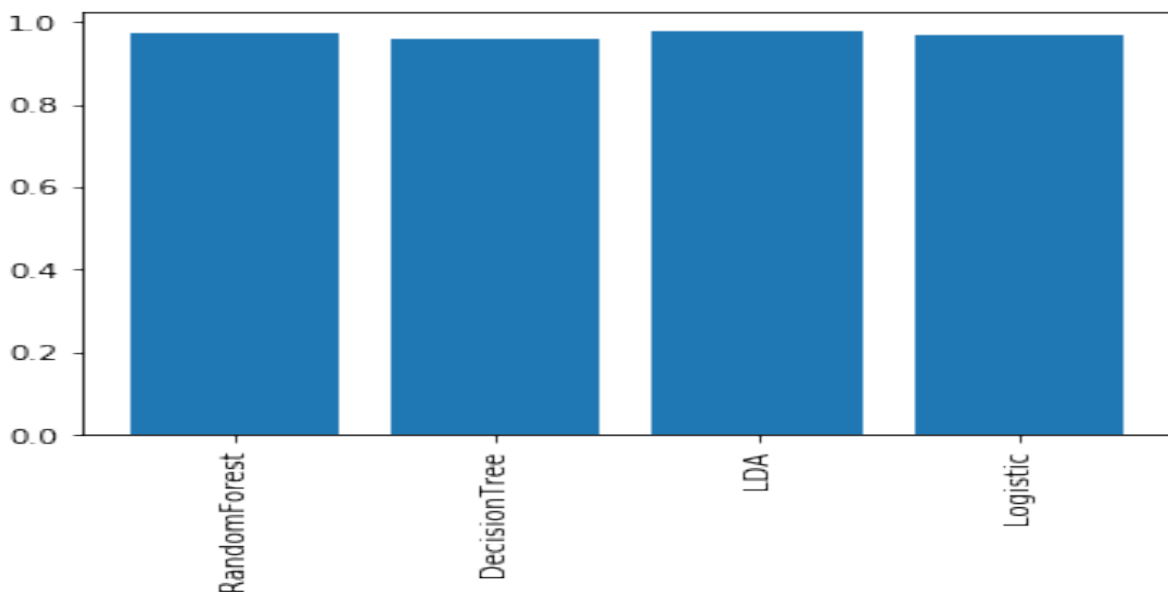
1.Training Set 1(X_{train_2})



results1

```
{'RandomForest': 0.9715059672031817,  
'DecisionTree': 0.9588452329612815,  
'LDA': 0.9767833211995308,  
'Logistic': 0.9662321683337118}
```

2.Training Set 2(X_{train_3})

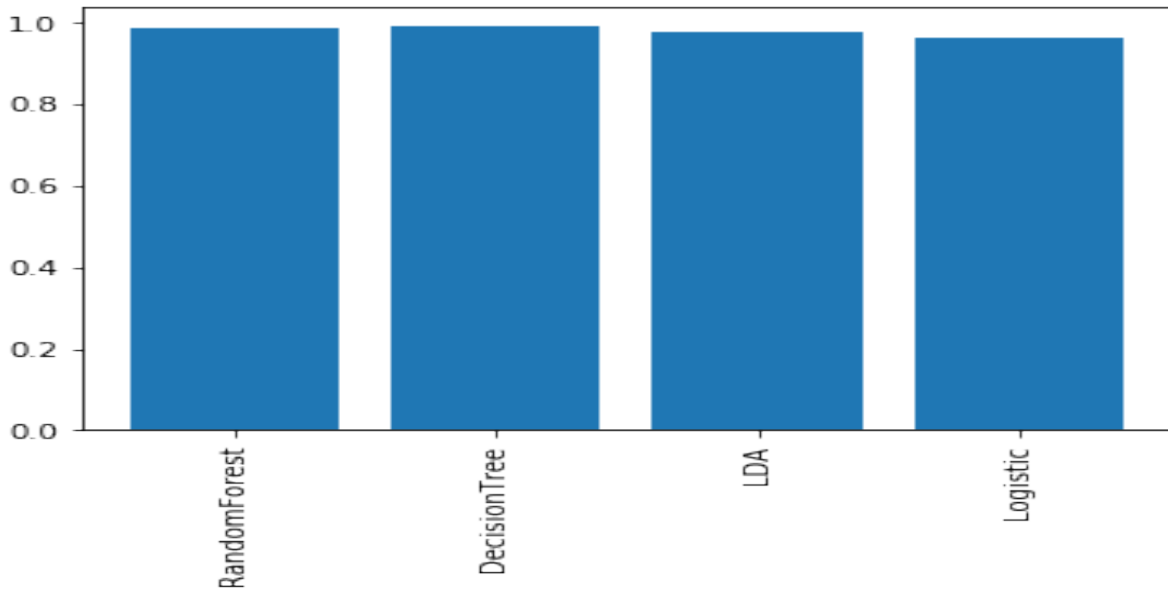


```

results2
{'RandomForest': 0.9669342286070239,
 'DecisionTree': 0.9588452329612815,
 'LDA': 0.9783662800852613,
 'Logistic': 0.9639456034673308}

```

3.Training Set 3(X_{train_4})



```

results3
{'RandomForest': 0.9838182990109019,
 'DecisionTree': 0.9771365153264613,
 'LDA': 0.9984168865435356,
 'Logistic': 0.9889194423706205}

```

It can be clearly seen that there is very slight differences of the accuracy of the various models. But it is to be noticed that the last method of feature selection provides with the best cross-validation results for all the models. So we shall go forward and train all the models on that data-set and check their respective accuracy scores , confusion matrix and computational. Only after reviewing all these , we shall try to converge on the best model for my data-set.

Model Comparison and Selection

Now we shall train the models on the training set (X_{train_4} , y_{train}) and score it by comparing the predicted y from X_{test_4} with the original y of the test set(y_{test}) . The confusion matrix from each of the models along with their computational time will also be considered while choosing the appropriate model.

1.Random Forest Classifier

```
%%time
run_randomforest (X_train_4 ,X_test_4 , y_train ,y_test)
Accuracy on test set: 0.9819524200164069
[[1240    17]
 [   27 1154]]
Wall time: 789 ms
```

2.Decision Tree Classifier

```
%%time
run_decisiontree(X_train_4 , X_test_4 , y_train , y_test)
Accuracy on test set: 0.9745693191140279
[[1211    46]
 [   16 1165]]
Wall time: 31 ms
```

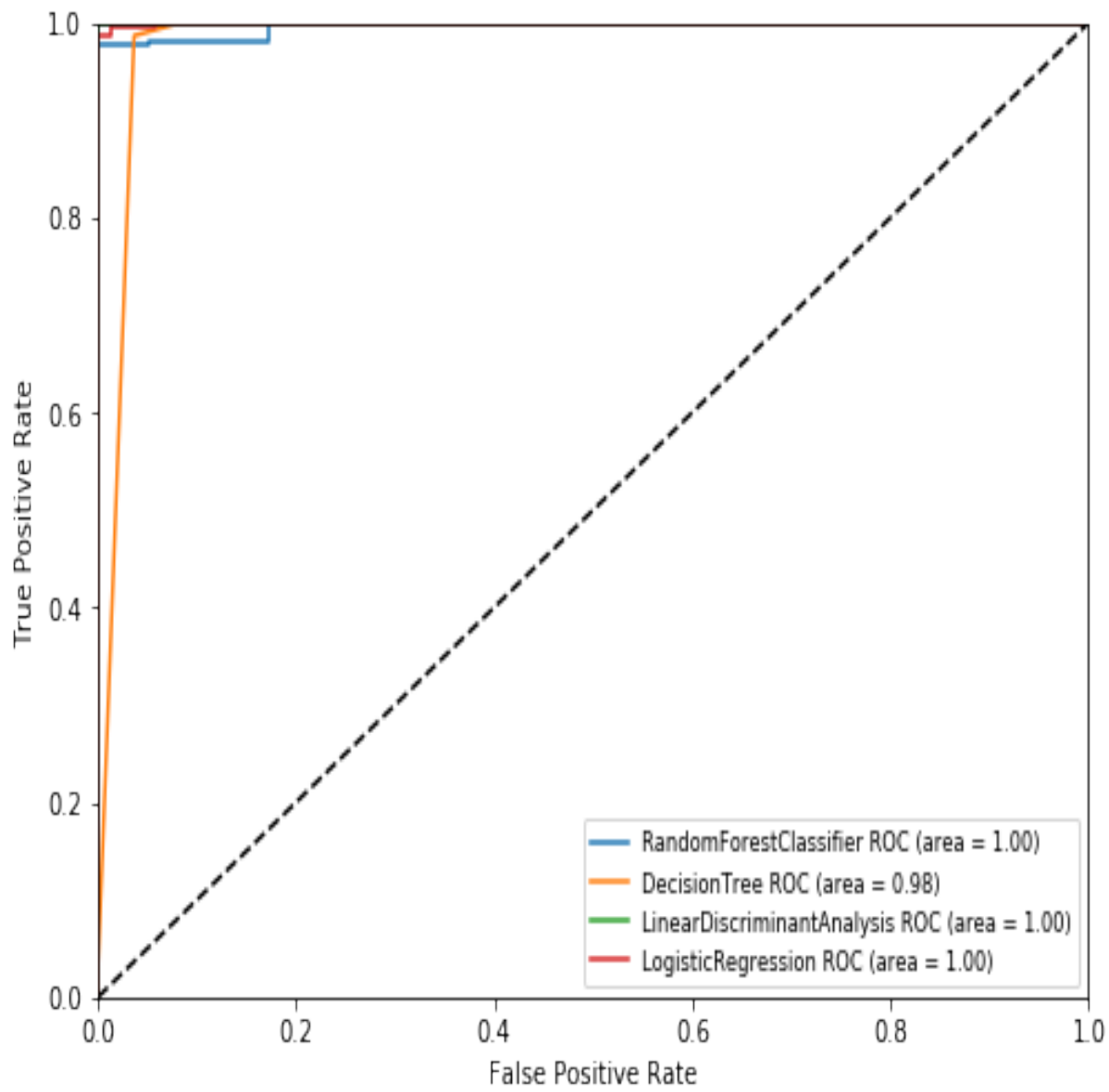
3.Logistic Regression

```
%%time
run_logistic(X_train_4 , X_test_4 , y_train , y_test)
Accuracy on test set: 0.9987694831829368
[[1257     0]
 [     3 1178]]
Wall time: 153 ms
```

4.Linear Discriminant Analysis

```
%%time
run_LDA(X_train_4 , X_test_4 , y_train , y_test)
Accuracy on test set: 0.9864643150123051
[[1240    17]
 [   16 1165]]
Wall time: 37.1 ms
```

From the above accuracy scores , confusion matrices and computational time we can compare the models minutely. An ROC-AUC curve of all the models, provided below , helps in the comparative study.



Conclusion

Of all the models ,the **logistic model** fits the best with an **accuracy score** of 0.9987694831829368. So we may like to think that this must be the best model.

But there is a major drawback of the model that cannot be overlooked : the feature selection method that we chose to be the best is based on a logistic regression model. So it is natural that the features thus selected will provided the best result on the test set also.It will be a kind of an over-fitting of data.

Again the computational time of the **Random Forest Classifier model** is the **maximum(789ms)**. Though it seems to quite small , we have to keep in mind that in the real-world datasets will have millions , even billions of instances. When applied to that dataset the computational time will be huge.

Thus we will not consider the logistic model or the random forest classifier to be the ideal ones.

Of the remaining two , the **Decision Tree classifier** has the **minimum computational time(31ms)** and the **Linear Discriminant Analysis classifier** has the **maximum accuracy(0.9864643150123051)** .

Keeping in mind that the large amount of data that will be classified by any model in the real world, **I chose Linear Discriminant Analysis classifier as the best model**. Moreover,the accuracy that the Linear Discriminant Analysis classifier provided was more than enough and the computational time difference is very small(6.1ms). A little room for error is left to deal with various discrepancies that may occur.

Appendix:

Python Code:

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns


from sklearn.model_selection import train_test_split , cross_val_score

from sklearn.ensemble import RandomForestClassifier

from sklearn.tree import DecisionTreeClassifier

from sklearn.linear_model import LogisticRegression

from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

from sklearn.metrics import accuracy_score , confusion_matrix ,roc_curve, auc

from sklearn.feature_selection import chi2 , SelectPercentile

from sklearn.feature_selection import mutual_info_classif ,SelectFromModel

from sklearn.feature_selection import VarianceThreshold


#Import of Data

df=pd.read_csv ('datasets_478_974_mushrooms.csv')

df.head()

df.columns


#Data Pre-Processing

#Null-value and Missing value Detection

for attr in df.columns:

    #Format subplots

    fig, ax = plt.subplots(1,2)

    plt.subplots_adjust(right=2)
```

```
#Construct values to count in each column
```

```
a=set(df[df['stalk-root']=='?'][attr])
```

```
b=set(df[df['stalk-root']!='?'][attr])
```

```
c = a.union(b)
```

```
c = np.sort(np.array(list(c)))
```

```
#Build each subplot
```

```
sns.countplot(x=df[df['stalk-root']=='?'][attr], order=c, ax=ax[0]).set_title('stalk-root == ?')
```

```
sns.countplot(x=df[df['stalk-root']!='?'][attr], order=c, ax=ax[1]).set_title('stalk-root != ?')
```

```
#Plot the plots
```

```
fig.show()
```

```
print( (len(df[df['stalk-root']=='?']) / len(X))*100, '%', sep="")
```

```
#This shows that there is no other null values
```

```
df.isnull().sum()
```

```
#Imbalance Check
```

```
# As the y takes both p(poisonous) and e(edible)
```

```
 #(the two classes of the y present in the dataset) almost equally , we can
```

```
# conclude that there is not an imbalanced dataset.
```

```
df.groupby('class').size()
```

```
#Data Transformation
```

```
for col in df.columns:
```

```
    df = pd.get_dummies(df,prefix=col, columns = [col], drop_first=True)
```

```
df.head()
```

```
#Formation of regressors and target
```

```
X = df.iloc[:,1:96]
```

```
y = df.iloc[:, 0]
```


X.shape

#Splitting of the dataset into training set and test set

```
X_train , X_test , y_train , y_test = train_test_split(X,y,test_size =0.3 , random_state= 42)
```

```
X_train.shape ,X_test.shape
```

#Data Reduction

Removal constant and duplicate features

```
filter=VarianceThreshold(threshold=0)
```

```
filter.fit(X_train)
```

```
X_trainf=filter.transform(X_train)
```

```
X_testf=filter.transform(X_test)
```

```
X_train_T=X_trainf.T
```

```
X_test_T=X_testf.T
```

```
X_train_T= pd.DataFrame(X_train_T)
```

```
X_test_T= pd.DataFrame(X_test_T)
```

```
dup_feat = X_train_T.duplicated()
```

```
feat_to_keep = [ not index for index in dup_feat]
```

```
X_train_1=X_train_T[feat_to_keep].T
```

```
X_test_1=X_test_T[feat_to_keep].T
```

```
X_train_1.shape , X_test_1.shape
```

#Judging the importance of the features

```
f_score = chi2(X_train_1 , y_train)
```

```
p_values = pd.Series (f_score[1], index=X_train_1.columns)
```

```
p_values.index = X_train_1.columns
```

```
p_values.sort_values(ascending = True , inplace = True)
```

```
p_values.plot.bar(figsize= (16,5))
```

#Feature selection based on three different metrics

```

def select_feat1(X_train , y_train ,X_test):
    fs=SelectPercentile(score_func=chi2,percentile=20)
    fs.fit(X_train ,y_train)
    X_train_fs=fs.transform(X_train)
    X_test_fs=fs.transform(X_test)
    return X_train_fs , X_test_fs

def select_feat2(X_train , y_train ,X_test):
    fs=SelectPercentile(score_func=mutual_info_classif ,percentile=20)
    fs.fit(X_train ,y_train)
    X_train_fs=fs.transform(X_train)
    X_test_fs=fs.transform(X_test)
    return X_train_fs , X_test_fs

def select_feat3(X_train , y_train , X_test):
    sel=SelectFromModel(LogisticRegression(penalty='l1', C=0.5 , random_state = 42,
                                             solver='liblinear' , n_jobs=-1 ) , max_features = 20 )
    sel.fit(X_train_1,y_train)
    sel.get_support()
    sel.estimator_.coef_
    X_train_l1=sel.transform(X_train_1)
    X_test_l1=sel.transform(X_test_1)
    return X_train_l1,X_test_l1

X_train_2 , X_test_2 =select_feat1(X_train_1,y_train , X_test_1)
X_train_3 , X_test_3 =select_feat2(X_train_1,y_train , X_test_1)
X_train_4 , X_test_4 =select_feat3(X_train_1,y_train , X_test_1)
X_train_2.shape , X_train_3.shape ,X_train_4.shape

#Model Comparison

```

```
clf1=RandomForestClassifier (n_estimators=100 ,max_depth = 3 ,
                             criterion="entropy",random_state=42 , n_jobs=-1 )
clf2=DecisionTreeClassifier(criterion="entropy",max_depth = 3 ,random_state=42 )
clf3=LogisticRegression(penalty='elasticnet', C=0.5 , random_state = 42, solver='saga',l1_ratio=0.5,
                          n_jobs=-1 )
clf4=LinearDiscriminantAnalysis(solver = 'eigen' , shrinkage = 'auto')
```

```
def run_randomforest (X_train , X_test , y_train , y_test):
    clf =clf1
    clf.fit(X_train , y_train)
    y_pred = clf.predict(X_test)
    print('Accuracy on test set:', accuracy_score(y_test , y_pred))
    print(confusion_matrix(y_test, y_pred))
```

```
def run_decisiontree(X_train , X_test , y_train , y_test):
    clf = clf2
    clf.fit(X_train , y_train)
    y_pred = clf.predict(X_test)
    print('Accuracy on test set:', accuracy_score(y_test , y_pred))
    print(confusion_matrix(y_test, y_pred))
```

```
def run_logistic(X_train , X_test , y_train , y_test):
    clf =clf3
    clf.fit(X_train , y_train)
    y_pred = clf.predict(X_test)
    print('Accuracy on test set:', accuracy_score(y_test , y_pred))
    print(confusion_matrix(y_test, y_pred))
```

```
def run_LDA(X_train , X_test , y_train , y_test):
    clf =clf4
    clf.fit(X_train , y_train)
```

```
y_pred = clf.predict(X_test)
print('Accuracy on test set:', accuracy_score(y_test , y_pred))
print(confusion_matrix(y_test, y_pred))
```

Cross-validation comparison

```
results1= {}
results1['RandomForest']=cross_val_score (clf1 , X_train_2 , y_train , cv=5).mean()
results1['DecisionTree']=cross_val_score (clf2 , X_train_2 , y_train , cv=5).mean()
results1['LDA']=cross_val_score (clf3 , X_train_2 , y_train , cv=5).mean()
results1['Logistic']=cross_val_score (clf4 , X_train_2 , y_train , cv=5).mean()
plt.bar(range(len(results0)), results0.values(), align='center')
plt.xticks(range(len(results0)), list(results0.keys()), rotation='vertical')
plt.show()
```

results1

```
results2= {}
results2['RandomForest']=cross_val_score (clf1 , X_train_3 , y_train , cv=5).mean()
results2['DecisionTree']=cross_val_score (clf2 , X_train_3 , y_train , cv=5).mean()
results2['LDA']=cross_val_score (clf3 , X_train_3 , y_train , cv=5).mean()
results2['Logistic']=cross_val_score (clf4 , X_train_3 , y_train , cv=5).mean()
plt.bar(range(len(results1)), results1.values(), align='center')
plt.xticks(range(len(results1)), list(results1.keys()), rotation='vertical')
plt.show()
```

results2

```
results3= {}
results3['RandomForest']=cross_val_score (clf1 , X_train_4 , y_train , cv=5).mean()
results3['DecisionTree']=cross_val_score (clf2 , X_train_4 , y_train , cv=5).mean()
```

```

results3['LDA']=cross_val_score (clf3 , X_train_4 , y_train , cv=5).mean()
results3['Logistic']=cross_val_score (clf4 , X_train_4 , y_train , cv=5).mean()
plt.bar(range(len(results)), results.values(), align='center')
plt.xticks(range(len(results)), list(results.keys()), rotation='vertical')
plt.show()

```

results3

#ROC-AUC Curve ,Accuracy Score , Confusion matrix and Computational Time

```

models = [

    {
        'label' : 'RandomForestClassifier',
        'model': clf1,
        'roc_Xtrain': X_train_4,
        'roc_Xtest': X_test_4,
        'roc_ytrain': y_train,
        'roc_ytest': y_test,
    },
    {
        'label' : 'DecisionTree',
        'model': clf2,
        'roc_Xtrain': X_train_4,
        'roc_Xtest': X_test_4,
        'roc_ytrain': y_train,
        'roc_ytest': y_test,
    },
    {
        'label' : 'LinearDiscriminantAnalysis',
        'model': clf3,

```

```

        'roc_Xtrain': X_train_4,
        'roc_Xtest': X_test_4,
        'roc_ytrain': y_train,
        'roc_ytest': y_test,
    },
    {
        'label' : 'LogisticRegression',
        'model': clf4,
        'roc_Xtrain': X_train_4,
        'roc_Xtest': X_test_4,
        'roc_ytrain': y_train,
        'roc_ytest': y_test,
    }
]

```

```
plt.clf()
```

```
plt.figure(figsize=(8,6))
```

```
for m in models:
```

```
    m['model'].probability = True
```

```
    probas = m['model'].fit(m['roc_Xtrain'], m['roc_ytrain']).predict_proba(m['roc_Xtest'])
```

```
        fpr, tpr, thresholds = roc_curve(m['roc_ytest'], probas[:, 1])
```

```
    roc_auc = auc(fpr, tpr)
```

```
    plt.plot(fpr, tpr, label='%s ROC (area = %0.2f)' % (m['label'], roc_auc))
```

```
plt.plot([0, 1], [0, 1], 'k--')
```

```
plt.xlim([0.0, 1.0])
```

```
plt.ylim([0.0, 1.0])
```

```
plt.xlabel('False Positive Rate')
```

```
plt.ylabel('True Positive Rate')
plt.legend(loc=0, fontsize='small')
plt.show()
```

```
%%time
run_randomforest(X_train_4,X_test_4,y_train,y_test)

%%time
run_decisiontree(X_train_4,X_test_4,y_train,y_test)

%%time
run_logistic(X_train_4,X_test_4,y_train,y_test)

%%time
run_LDA(X_train_4,X_test_4,y_train,y_test)
```